

INTRO TO SCIENTIFIC COMPUTING IN PYTHON

LECTURE 4

Zan Ahmad

Department of Applied Mathematics and Statistics,
Johns Hopkins University

January 15, 2026

Part I

SciPy Deep Dive: Algorithms for Scientific Problems

WHAT THIS LECTURE IS ABOUT

So far in this course:

NumPy gave us efficient numerical objects — arrays, matrices, and vectorized operations.

SciPy builds on NumPy by providing *algorithms* for problems that do not have closed-form solutions.

Today, we focus on three fundamental scientific problem types:

1. Finding unknown parameters (root finding / optimization)
2. Modeling time-evolving systems (ODEs)
3. Reconstructing continuous functions from data (interpolation)

Key idea:

SciPy turns mathematical problems into reliable numerical solutions.

ROOT FINDING AS A CORE SCIENTIFIC TASK

Many scientific problems reduce to solving equations of the form:

$$f(x) = 0$$

Examples include:

- ▶ equilibrium points in physics,
- ▶ steady states in biology,
- ▶ optimality conditions in optimization.

When analytic solutions do not exist, we must solve these equations numerically.

BRACKETING ROOTS

Suppose we have a continuous function $f(x)$.

If we can find an interval $[a, b]$ such that:

$$f(a)f(b) < 0,$$

then by the Intermediate Value Theorem, a root exists in (a, b) .

This mathematical guarantee is the foundation of many robust numerical solvers.

ROOT FINDING IN SCI-PY: BRENT'S METHOD

```
from scipy import optimize

def f(x):
    return x**3 - x - 2

sol = optimize.root_scalar(
    f,
    bracket=[1, 2],
    method="brentq"
)

sol.root
```

SciPy returns both the solution and diagnostic information about convergence.

HOW BRENT'S METHOD WORKS

Brent's method combines three ideas:

- ▶ **Bisection** — guaranteed convergence
- ▶ **Secant steps** — faster convergence
- ▶ **Safety checks** — prevent divergence

At each iteration, the algorithm chooses the fastest step that still guarantees the root remains bracketed.

Key idea:

Brent's method is fast when it can be, and safe when it must be.

BISECTION VS. SECANT (MATHEMATICAL INSIGHT)

Bisection repeatedly halves an interval:

$$m = \frac{a + b}{2}$$

ensuring convergence but progressing slowly.

Secant approximates the root using a line:

$$x = b - f(b) \frac{b - a}{f(b) - f(a)}$$

which converges faster but can fail.

Brent's method adaptively switches between these strategies.

WHY ORDINARY DIFFERENTIAL EQUATIONS MATTER

Many scientific systems evolve over time according to laws of the form:

$$\frac{dx}{dt} = f(t, x)$$

Examples include:

- ▶ population dynamics,
- ▶ chemical kinetics,
- ▶ electrical circuits,
- ▶ neural activity.

These equations rarely have closed-form solutions.

INITIAL VALUE PROBLEMS

An ODE must be paired with an initial condition:

$$x(t_0) = x_0$$

The goal is to approximate the function $x(t)$ over a time interval.

This is known as an **initial value problem**.

SOLVING ODEs WITH SOLVE_IVP

```
from scipy.integrate import solve_ivp

def f(t, x):
    return 0.5 * x

sol = solve_ivp(
    f,
    t_span=(0, 10),
    y0=[1.0],
    method="RK45",
    dense_output=True
)
```

The solver automatically chooses time steps and controls error.

RUNGE–KUTTA METHODS

Runge–Kutta methods approximate solutions by sampling slopes inside each time step.

Instead of using a single slope:

$$x_{n+1} = x_n + hf(t_n, x_n),$$

they combine multiple slope estimates to improve accuracy.

ADAPTIVE TIME STEPPING IN RK45

RK45 computes two approximations per step:

- ▶ a 5th-order solution,
- ▶ a 4th-order solution.

The difference estimates local error:

$$\text{error} \approx \|x^{(5)} - x^{(4)}\|$$

The solver adjusts the step size to balance accuracy and efficiency automatically.

WHY INTERPOLATION IS NEEDED

Experimental and simulation data are sampled at discrete points.

Interpolation constructs a continuous function that:

- ▶ passes through all data points,
- ▶ can be evaluated at any intermediate location.

This is distinct from curve fitting, which allows approximation.

CUBIC SPLINE INTERPOLATION IN SCIPY

```
from scipy.interpolate import CubicSpline  
  
x = [0, 1, 2, 3]  
y = [0, 1, 0, 1]  
  
cs = CubicSpline(x, y, bc_type="natural")  
cs(1.5)
```

The result is a smooth function that interpolates the data exactly.

HOW CUBIC SPLINES WORK

A cubic spline is a piecewise polynomial function:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

The spline is constructed so that:

- ▶ the function is continuous,
- ▶ first derivatives are continuous,
- ▶ second derivatives are continuous.

NATURAL BOUNDARY CONDITIONS

To uniquely determine the spline, boundary conditions are required.

A **natural cubic spline** imposes:

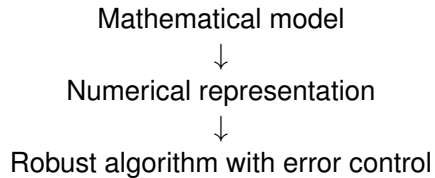
$$S''(x_0) = 0, \quad S''(x_n) = 0$$

This produces the smoothest interpolant consistent with the data.

Internally, SciPy solves a tridiagonal linear system to compute the spline.

ONE PATTERN ACROSS ALGORITHMS

Across root finding, ODEs, and interpolation, the workflow is the same:



SciPy provides the algorithmic bridge between theory and computation.

WHAT COMES NEXT

Next lecture or lab, we will:

- ▶ solve coupled ODE systems,
- ▶ explore optimization in higher dimensions,
- ▶ analyze numerical error and stability.

Goal:

Develop intuition for choosing the right numerical tool.